

RobotFramework_TestsuitesManagement

v. 0.9.0

Mai Dinh Nam Son

14.11.2025

Contents

1	Introduction	1
2	Description	2
2.1	Meaning of "Test Suites Management"	2
2.2	Content of configuration files	3
2.2.1	Variant configuration file	3
2.2.2	Parameter configuration files	4
2.3	Access to configuration files	6
2.4	Activation of "Test Suites Management"	7
2.5	Variants selection	8
2.6	Local configuration	9
2.7	Priority of configuration parameters	10
2.8	Nested configuration files	11
2.9	Overwritten parameters	12
2.10	Naming convention	13
2.11	Version control interfaces	14
2.11.1	RobotFramework AIO interface for version control	14
2.11.2	Explicit version control of the RobotFramework AIO	14
2.11.3	Explicit version control of the TestsuitesManagement	15
2.11.4	User defined version control	15
2.11.5	Python interface for version control	17
3	The JSONP format	18
3.1	Standard JSON format	18
3.2	Boolean and null values	20
3.3	Comments	21
3.4	Import of JSON files	22
3.5	Overwriting parameters	25
3.6	dotdict notation	34
3.7	Dynamic key names	36
3.8	Implicit creation of dictionaries	37
3.9	Python inline code	39
3.10	Special characters within key names	40
3.11	Byte sequences	41
4	CConfig.py	42
4.1	Class: CConfig	42
4.1.1	Method: loadCfg	42

4.1.2	Method: checkVersion	43
5	CKeywords.py	44
5.1	Class: CSetupKeywords	44
5.1.1	Keyword: testsuite_setup	44
5.1.2	Keyword: testsuite_teardown	44
5.1.3	Keyword: testcase_setup	44
5.1.4	Keyword: testcase_teardown	44
5.2	Class: CGeneralKeywords	44
5.2.1	Keyword: get_config	45
5.2.2	Keyword: load_json	45
5.2.3	Keyword: is_robotframework_aio	45
5.2.4	Keyword: get_package_context_file	45
5.2.5	Keyword: get_reference_version	45
5.2.6	Keyword: get_reference_version_date	45
5.2.7	Keyword: get_reference_app_name	46
5.2.8	Keyword: get_reference_installer_location	46
5.2.9	Keyword: get_tsm_version	46
5.2.10	Keyword: get_tsm_version_date	46
5.2.11	Keyword: get_tsm_app_name	46
5.2.12	Keyword: get_tsm_installer_location	46
5.2.13	Keyword: get_bundle_version	46
5.2.14	Keyword: get_bundle_version_date	46
5.2.15	Keyword: get_bundle_name	46
5.2.16	Keyword: get_bundle_installer_location	46
5.2.17	Keyword: check_version	46
5.2.18	Keyword: get_status_messages	46
6	COnFailureHandle.py	47
6.1	Class: COnFailureHandle	47
6.1.1	Method: is_noney	47
7	CStruct.py	48
7.1	Class: CStruct	48
8	CVersion.py	49
8.1	Class: enVersionCheckResult	49
8.2	Class: StatusMessages	49
8.2.1	Method: get_messages_dict	49
8.2.2	Method: get	49
8.2.3	Method: set	49
8.3	Class: CVersion	49
8.3.1	Method: verifyVersion	50
8.3.2	Method: checkVersion	50
8.3.3	Method: get_last_error	51
8.3.4	Method: bValidateMinVersion	51
8.3.5	Method: bValidateMaxVersion	51

8.3.6	Method: bValidateSubVersion	51
8.3.7	Method: tupleVersion	52
9	Event.py	53
9.1	Class: Event	53
9.1.1	Method: trigger	53
10	ScopeEvent.py	54
10.1	Class: ScopeEvent	54
10.1.1	Method: trigger	54
10.2	Class: ScopeStart	54
10.3	Class: ScopeEnd	54
11	__init__.py	55
11.1	Function: on	55
11.2	Function: dispatch	55
11.3	Function: register_event	55
12	LibListener.py	56
12.1	Class: LibListener	56
13	app_config.py	57
13.1	Class: AppConfig	57
13.1.1	Method: is_robotframework_ao	57
13.1.2	Method: get_package_context_file	57
13.1.3	Method: get_reference_version	57
13.1.4	Method: get_reference_version_date	57
13.1.5	Method: get_reference_app_name	57
13.1.6	Method: get_reference_installer_location	57
13.1.7	Method: get_tsm_version	57
13.1.8	Method: get_tsm_version_date	58
13.1.9	Method: get_tsm_app_name	58
13.1.10	Method: get_tsm_installer_location	58
13.1.11	Method: get_bundle_version	58
13.1.12	Method: get_bundle_version_date	58
13.1.13	Method: get_bundle_name	58
13.1.14	Method: get_bundle_installer_location	58
14	app_logger.py	59
14.1	Function: setup_app_logger	59
15	__init__.py	60
15.1	Class: RobotFramework_TestsuitesManagement	60
15.1.1	Method: run_keyword	60
15.1.2	Method: get_keyword_tags	60
15.1.3	Method: get_keyword_documentation	60
15.1.4	Method: failure_occurred	60
15.2	Class: CConfigLevel	60
15.3	Class: CTestsuitesCfg	60

16 Appendix	61
17 History	62

Chapter 1

Introduction

The **RobotFramework_TestsuitesManagement** enables users to define dynamic configuration values within separate configuration files in JSON format.

These configuration values are available during test execution - but under certain conditions that can be defined by the user (e.g. to realize a variant handling). This means: Not all parameter values are available during test execution - only the ones that belong to the current test scenario.

To realize this, the **RobotFramework_TestsuitesManagement** provides the following features:

1. Split all possible configuration values into several JSON configuration files, with every configuration file contains a specific set of values for configuration parameter
2. Use nested imports of JSON configuration files
3. Follow up definitions in configuration files overwrite previous definitions (of the same parameter)
4. Select between several criteria to let the Robot Framework use a certain JSON configuration file

How to install

The **RobotFramework_TestsuitesManagement** can be installed in two different ways: via PyPi (recommended for users) and via GitHub (recommended for developers).

Installation details can be found in the [README](#).

Further links

For self-study a tutorial is available containing lots of examples. Here you find the rendered [tutorial documentation](#).

For the development environment **VSCodium** an extension is available to support the features of the **RobotFramework_TestsuitesManagement**: [vscode-jsonp](#). This extension adapts e.g. the syntax highlighting of the editor.

Chapter 2

Description

2.1 Meaning of "Test Suites Management"

In the scope of the Robot Framework a test suite is either a single robot file containing one or more test cases, or a set of several robot files.

Usually all test cases of a test suite run under the same conditions - but these conditions may be different. For example the same test case is used to test several different variants of a system under test. Every variant requires individual values for certain configuration parameters.

Tests are carried out at several test benches. All test benches have different hardware configurations. Also the different test benches may require individual values for configuration parameters used in the tests.

Therefore the same tests have to run under different conditions!

The Robot Framework provides several places to define parameters: robot files, resource files, parameter files. But these parameters are fixed. Therefore we need a more dynamic way of accessing parameters. And we postulate the following: When switching between tests of several variants and test executions on several test benches, no changes shall be required within the test code.

The outcome is that another position has to be introduced to store values for variant and test bench specific parameters. And a possibility has to be provided to dynamically make either the one or the other set of values available during the execution of tests - depending on outer circumstances like "*which variant?*" and "*which test bench?*". Those dynamic configuration values are stored within separate configuration files in JSON format and the **RobotFramework.TestsuitesManagement** makes the values available globally during the test execution.

Two different kinds of JSON configuration files are involved:

1. *parameter configuration files*

These configuration files contain all parameter definitions (can be more than one configuration file in a project)

2. *variant configuration file*

This is a single configuration file containing the mapping between the several parameter configuration files and a name (usually the name of a variant). This name can be used in command line to select a certain parameter configuration file containing the values for this variant.

Background: It's easier simply to use a name for referencing a certain variant instead of having the need always to mention the path and name of a configuration file.

To realize a concrete test suites management for your project, you need to

- identify the parameters that are variant specific, depending on the number of variants in your project,
- identify the parameters that are test bench specific, depending on the number of test benches in your project,
- identify the parameters that are both: variant specific and test bench specific,
- identify the parameters that have the same value in all variants and test benches.

After this

- for every set of parameters (variant specific and bench specific) you have to introduce a certain parameter configuration file,
- in the variant configuration file you have to define for every variant a variant name together with the path to the corresponding parameter configuration file.

Basically all configuration files of the **RobotFramework.TestsuitesManagement** are implemented in JSON format. This format is extended by some useful features like code comments and imports (nested configuration files). This is explained in more detail in the following chapters. These features cause deviations from standard JSON format. To give applications like editors or syntax checkers a chance to handle these deviations (without invalid findings), all JSON configurations files of the **RobotFramework.TestsuitesManagement** have the extension `.jsonp`, instead of `.json`.

The content of the configuration files is described in the next section.

2.2 Content of configuration files

2.2.1 Variant configuration file

This file configures the access to all variant dependent `robot_config*.jsonp` files.

```
{
  "default": {
    "name": "robot_execution_config.jsonp",
    "path": "./config/"
  },
  "variant_1": {
    "name": "robot_config_variant_1.jsonp",
    "path": "./config/"
  },
  "variant_2": {
    "name": "robot_config_variant_2.jsonp",
    "path": "./config/"
  },
  "variant_3": {
    "name": "robot_config_variant_3.jsonp",
    "path": "./config/"
  }
}
```

The example above contains definitions for three variants with names:

`variant_1`, `variant_2` and `variant_3`. Additionally a variant named `default` is defined. This default configuration becomes active in case of no certain variant name is provided when the test suite is being executed.

The paths to the `robot_config*.jsonp` files could be absolute paths or relative to the position of the mapping file.

It is of course still possible to use the standard notation for relative paths:

```
"path": "./config/"
```

2.2.2 Parameter configuration files

In these configuration files all parameters are defined, that shall be available globally during test execution.

Some parameters are required. Optionally the user can add own ones. The following example shows the smallest version of a parameter configuration file containing only the most important parameters. This version is a default version and part of the **RobotFramework.TestsuitesManagement** installation.

```
{
  "WelcomeString"    : "Hello... Robot Framework is running now!",
  "Maximum_version"  : "1.0.0",
  "Minimum_version"  : "0.6.0",
  "Project"          : "RobotFramework Testsuites",
  "TargetName"       : "Device_01"
}
```

`Project`, `WelcomeString` and `TargetName` are simple strings that can be used anyhow. `Maximum_version` and `Minimum_version` are part of a version control mechanism: In case of the version of the currently installed software is outside the range between `Minimum_version` and `Maximum_version`, the test execution stops with an error message.

What is the meaning of "currently installed software"?

- The first possibility is that the **RobotFramework.TestsuitesManagement** runs stand-alone, that means, it is not part of a larger bundle (like the RobotFramework AIO). The installation from PyPi or GitHub causes such a stand-alone installation. In this case the component version of the **RobotFramework.TestsuitesManagement** itself is used for a version control against `Minimum_version` and `Maximum_version`.
- The second possibility is that the **RobotFramework.TestsuitesManagement** runs as part of the RobotFramework AIO. In this case the version of the entire RobotFramework AIO is used for a version control instead.

The version control mechanism is optional. In case you do not need to have your tests under version control, you can set the versions to the value `null`.

```
"Maximum_version" : null,
"Minimum_version" : null,
```

As an alternative it is also possible to remove `Minimum_version` and `Maximum_version` completely.

In case you define only one single version number, only this version number is considered. The following combination makes sure, that the installed software at least is of version 0.6.0, but there is no upper version limit:

```
"Maximum_version" : null,
"Minimum_version" : "0.6.0",
```

Hint: The parameters are keys of an internal configuration dictionary. They have to be accessed in the following way:

```
Log    Maximum_version : ${CONFIG}[Maximum_version]
Log    Project : ${CONFIG}[Project]
```

The following example is an extended version of a configuration file containing also some user defined parameters.

```
{
  "WelcomeString"    : "Hello... Robot Framework is running now!",
  "Maximum_version"  : "1.0.0",
  "Minimum_version"  : "0.6.0",
  "Project"          : "RobotFramework Testsuites",
  "TargetName"       : "Device_01"
  "params": {
    // global parameters
    "global" : {
      "param1" : "ABC",
      "param2" : 25
    }
  }
}
```

User defined parameters have to be placed inside `params:global`. The intermediate level `global` is introduced to enable further parameter scopes than `global` in future.

All user defined parameters have the scope `params:global` per default. Therefore they can be accessed directly:

```
Log    param1 : ${param1}
```

And another feature can be seen in the example above:

In the context of the **RobotFramework-TestsuitesManagement** the JSON format is an extended one. Deviating from JSON standard it is possible to comment out lines with starting them with a double slash `//`. This allows to add explanations about the meaning of the defined parameters already within the JSON file.

2.3 Access to configuration files

With an installed **RobotFramework.TestsuitesManagement** every test execution requires a configuration - that is the accessibility of a configuration file in JSON format. The **RobotFramework.TestsuitesManagement** provides four different possibilities - also called *level* - to realize such an access. These possibilities are sorted and the **RobotFramework.TestsuitesManagement** tries to access the configuration file in a certain order: Level 1 has the highest priority and level 4 has the lowest priority.

Level 1

Path and name of a parameter configuration file is provided in command line of the Robot Framework.

Level 2 (recommended)

The name of the variant is provided in command line of the Robot Framework.

This level requires that a variant configuration file is passed to the suite setup of the **RobotFramework.TestsuitesManagement**.

Level 2 includes the automated selection of a default variant (in case of no variant name is provided in command line). Also this default variant has to be defined within the variant configuration file.

Level 3

The **RobotFramework.TestsuitesManagement** searches for parameter configuration files within a folder `config` in current test suite folder. In case of such a folder exists and parameter configuration files are inside, they will be used.

Level 4 (unwanted, fallback solution only)

The **RobotFramework.TestsuitesManagement** uses the default configuration file that is part of the installation.

Summary

- With highest priority a parameter configuration file provided in command line, is considered - even in case of also other configuration files (level 2 - level 4) are available.
- If a parameter configuration file is not provided in command line, but a variant name, then the configuration belonging to this variant, is loaded - even in case of also other configuration files (level 3 - level 4) are available.
- If nothing is specified in command line, then the **RobotFramework.TestsuitesManagement** tries to find parameter configuration files within a `config` folder and take them if available - even in case of also the level 4 configuration file is available.
- In case of the user does not provide any information about parameter configuration files to use, the **RobotFramework.TestsuitesManagement** loads the default configuration from installation folder (fallback solution; level 4).

In this context two aspects are important to know for users:

1. *Which parameter configuration file is selected for the test execution?*
To answer this question the log file contains the path and the name of the selected parameter configuration file.
2. *For which reason is this parameter configuration file selected?*
To answer this question the log file also contains the level number. The level number indicates the reason.

With these log file entries the test execution is clearly understandable, traceable and scales for huge test suites.

Why is level 2 the recommended one?

Level 2 is the most flexible and extensible solution. Because the robot files contain a link to a variants configuration file, the possible sets of parameter values can already be taken out of the code.

The values selected by level 1, you only see in the log files, but not in the code, because the selection happens in command line only.

Level 3 has a rather strong binding between robot files and configuration files. If you start the test implementation based on level 3 and after this want to have a variant handling, then you have to switch from level 3 to level 2 - and this causes effort in implementation.

Whereas if you start with level 2 immediately and need to consider another set of configuration values for the same tests, then you only have to add another parameter configuration file and another entry in the variants configuration file, without changing any test implementation.

We strongly recommend not to mix up several different configuration levels in one project!

2.4 Activation of "Test Suites Management"

To activate the test suites management you have to import the **RobotFramework.TestsuitesManagement** library in the following way:

```
Library      RobotFramework.TestsuitesManagement    WITH NAME    tm
```

We recommend to use the `WITH NAME` option to shorten the robot code a little bit.

The next step is to call the `testsuite_setup` of the **RobotFramework.TestsuitesManagement** within the `Suite Setup` of your test:

```
Suite Setup    tm.testsuite_setup
```

As long as you

- do not provide a parameter configuration file in command line when executing the test suite (level 1),
- do not provide a variants configuration file as parameter of the `testsuite_setup` (level 2),
- do not have a `config` folder containing parameter configuration files in your test suites folder (level 3),

the **RobotFramework.TestsuitesManagement** falls back to the default configuration (level 4).

In case you want to realize a variant handling you have to provide the path and the name of a variants configuration file to the `testsuite_setup` :

```
Suite Setup    tm.testsuite_setup    .../config/exercise_variants.jsonp
```

The path to the variants configuration file can be absolute path or relative to the position of the robot file.

Another aspect is important: the **three dots**. The path to the `/config/exercise_variants.jsonp` file depends on the robot file location. A different number of `../` is required dependent on the directory depth of the robot file location.

Therefore we use here three dots to tell the **RobotFramework.TestsuitesManagement** to search from the robot file location up till the `/config/exercise_variants.jsonp` file is found:

```
./config/exercise_variants.jsonp
../config/exercise_variants.jsonp
../../config/exercise_variants.jsonp
../../../../config/exercise_variants.jsonp
```

and so on.

Hint: You are free to move your test suites one or more level up or down in the file system, but using the three dots notation enables you to let the position of the `config` folder unchanged.

To ease the analysis of a test execution, the log file contains informations about the selected level and the path and the name of the used configuration file, for example:

```
Running with configuration level: 2
CfgFile Path: ./config/exercise_config.jsonp
```

Please consider: The `testsuite_setup` requires a variants configuration file (in the example above: `exercise_variants.jsonp`) - whereas the log file contains the resulting parameter configuration file (in the example above: `exercise_config.jsonp`), that is selected depending on the name of the variant provided in command line of the Robot Framework.

2.5 Variants selection

In a previous section the level concept for configuration files has been explained. This section contains corresponding code examples.

1. Selection of a certain parameter configuration file in command line

```
--variable config_file:"(path to parameter configuration file)"
```

2. Selection of a certain variant per name in command line

```
--variable variant:"(variant name)"
```

3. Parameter configuration taken from `config` folder

This `config` folder has to be placed in the same folder than the test suites.

Parameter configuration files within this folder are considered under two different conditions:

- The configuration file has the name `robot_config.jsonp`. That is a fix name predefined by the **Robot-Framework.TestsuitesManagement**.
- The configuration file has the same name than a robot file inside the test suites folder, e.g.:
 - Name of test suite file: `example.robot`
 - Path and name of corresponding parameter configuration file: `./config/example.jsonp`

With this rule it is possible to give every test suite in a certain folder an own individual configuration.

2.6 Local configuration

It might be required to execute tests on several different test benches with every test bench has it's own individual hardware that might require configuration parameter values that are test bench specific. This can be related to common configuration parameters and also to parameters that are variant specific. In the second case a configuration parameter is both variant specific *and* test bench specific.

The *local configuration* feature of the **RobotFramework.TestsuitesManagement** provides the possibility to define test bench specific configuration parameter values.

The meaning of *local* in this context is: placed on a certain test bench - and valid for this bench only.

Also this local configuration is based on configuration files in JSON format. These files are the last ones that are considered when the configuration is loaded. The outcome is that it is possible to define default values for test bench specific parameters in other configuration files - to be also test bench independent. And it is possible to use the local configuration to overwrite these default values with values that are specific for a certain test bench.

Important:

- Local configuration files are fragments only - and not a full configuration! Even so they need to follow the JSON syntax rules. This means, at least they have to start with an opening curly bracket and they have to end with a closing curly bracket.
- Local configuration files must not contain the mandatory top level parameters like the `WelcomeString` and others.

Using the local configuration feature is an option and the **RobotFramework.TestsuitesManagement** provides two ways to realize it:

1. *per command line*

Path and name of the local parameter configuration file is provided in command line of the Robot Framework with the following syntax:

```
--variable local_config:"(path to local configuration file)"
```

2. *per environment variable*

An environment variable named `ROBOT_LOCAL_CONFIG` exists and contains path and name of a local parameter configuration file.

The user has to create this environment variable!

This mechanism allows a user - without any command line extensions - automatically to refer on every test bench to an individual local configuration, simply by giving on every test bench this environment variable an individual value.

The command line has a higher priority than the environment variable. If both is available the local configuration is taken from command line.

Recommendation: *To avoid an accidental overwriting of local configuration files in version control systems we recommend to give those files names that are test bench specific.*

2.7 Priority of configuration parameters

In previous sections the level concept has been explained. This concept introduces four levels of priority that define, which of the possible sources of configuration parameters are processed. But there are other rules involved that influence the priority:

- The local configuration has higher priority than other parameter configurations
- The command line has higher priority than definitions within configuration files

Already in command line we have several possibilities to make settings:

- Set a parameter configuration file (with **RobotFramework.TestsuitesManagement** command line variable `config_file` , level 1)
- Set a variant name (with **RobotFramework.TestsuitesManagement** command line variable `variant` , level 2)
- Set a local configuration (with **RobotFramework.TestsuitesManagement** command line variable `local_config`)
- Set any other variables (directly with Robot Framework command line variable `--variable`)

And it is possible that in all four use cases the same parameters are used. Or in other words: It is possible to use the `--variable` mechanism to define a parameter that is also defined within a parameter configuration or within a local configuration - or in both together.

Finally this is the order of processing (with highest priority first):

1. Single command line variable (`--variable`)
2. Local configuration (`local_config`)
3. Variant specific configuration (`config_file` or `variant`)

Meaning:

1. Variant specific configuration is overwritten by local configuration
2. Local configuration is overwritten by single command line variable

What happens in case of a command line contains both a `config_file` and a `variant` ?

`config_file` is level 1 and `variant` is level 2. Level 1 has higher priority than level 2. Therefore `config_file` is the valid one. This does **not** mean that `config_file` overwrites `variant` ! In case of a certain level is identified (here: level 1), all other levels are ignored. The outcome is that - in this example - the `variant` has no meaning. Between different levels there is an *either or* relationship. And that is the reason for that it makes no sense to define both in command line, a `config_file` and a `variant` . The **RobotFramework.TestsuitesManagement** throws an error in this case.

But when additionally `--variable` is used to define a new value for a parameter that is already defined in one of the involved configuration files, then the configuration file value is overwritten by the command line value.

And even this is not all. The Robot Framework provides further possibilities to define parameters in command line, e.g. by `--variablefile` . `--variable` and `--variablefile` are Robot Framework mechanisms to define parameters, whereas `config_file` and `local_config` are corresponding **RobotFramework.TestsuitesManagement** mechanisms.

The rules behind all are: `--variable` overrules `--variablefile` . Robot Framework mechanisms overrule **RobotFramework.TestsuitesManagement** mechanisms.

To avoid the things becoming too much complicated, we urgently recommend not to mixup both mechanisms to define *different values for the same parameters* (but to overwrite only a single variable with `--variable` might be OK).

2.8 Nested configuration files

In case of a project requires more and more parameters, it makes sense to split the growing configuration file into smaller ones.

This means, at first we have to split all configuration parameters in

1. parameters that are specific for a certain variant,
2. common parameters that have the same value for all variants

Placing those common parameters in every single variant specific parameter configuration file would create a lot of redundancy. This would also complicate the maintenance.

The solution is to use the variant specific configuration files only for variant specific parameters and to put all common parameters in a separate configuration file. This common parameter file has to be imported in every variant specific parameter file.

The outcome is that still with the selection of a certain variant specific parameter file both types of parameters are available: the variant specific ones and the common ones.

This can be done in the following way:

For example we have the following variant specific configuration files:

```
config/config_variant1.jsonp
config/config_variant2.jsonp
```

Additionally we have a configuration file with common parameters:

```
config/config_common.jsonp
```

The import of `config_common.jsonp` into `config_variant1.jsonp` and into `config_variant2.jsonp` is possible in the following way:

```
"params" : {
  "global": {
    "[import]" : "./config_common.jsonp",
    "teststring" : "variant specific value"
  }
}
```

The key `[import]` indicates the import of another configuration file. The value of the key is the path and name of this file.

Imports can be nested. An imported configuration file is allowed to contain imports also.

The content of the importing file and the content of all imported files are merged. In case of duplicate parameter names follow up definitions overwrite previous definitions of the same parameter!

Important:

- All imported configuration files are fragments only - and not a full configuration! Even so they need to follow the JSON syntax rules. This means, at least they have to start with an opening curly bracket and they have to end with a closing curly bracket.
- Imported configuration files must not contain the mandatory top level parameters like the `WelcomeString` and others.

2.9 Overwritten parameters

Summarized the **RobotFramework_TestsuitesManagement** provides three different types of parameter configuration files to define parameters:

1. A full standard parameter configuration file containing at least the mandatory parameters and - as option - also user defined parameters
2. A parameter configuration file fragment that is imported in other configuration files by the `[import]` key
3. A local parameter configuration file that is also a fragment only, and accessed either by command line or environment variable

All types of configuration file can be used

1. to define new parameters
2. to overwrite already existing parameters

This possibility only belongs to user defined parameters with scope `params:global` !

Example:

1. *Define a new parameter:*

```
"params" : {  
    "global": {  
        "teststring" : "initial value"  
    }  
}
```

2. *Overwrite an already existing parameter:*

To overwrite a parameter is - after the initial definition - possible at any follow up position

- in the same configuration file or
- in other configuration files like the imported ones or
- in a local configuration file

With the following syntax:

```
${params}['global']['teststring'] : "new value"
```

The resulting value of a parameter at the end depends on the priority (computation order) described in previous sections of this description.

2.10 Naming convention

Keys that are defined within `params['global']` in JSONP configuration files become **Robot Framework** variables. Therefore, the key names must conform to **Robot Framework**'s naming conventions.

The naming convention is: Key names may only contain letters, digits and underscores, and must start with a letter. Internally this is realized by the following regex pattern: `^\p{L}[\p{L}\p{Nd}_]*$`.

Example

- Valid key names are: `"abcParam"`, `"abcParam01"`, `"abc_Param_01"`, ...
- Invalid key names are: `"+param01"`, `"param$01"`, `"abc#Param"`, `"01abc_Param"`, ...

Caution



The **Robot Framework** ignores underscores. Nevertheless, underscores are allowed inside variable names, but you must be aware of the consequences when using them! The following code example demonstrates the effects.

Example

```
"params" : {
    "global": {
        "ABC"      : "A",
        "ABC__"    : "B",
        "A__B__C"  : "C"
    }
}
```

With

```
Log    ABC: ${ABC}
Log    ABC__: ${ABC__}
Log    A__B__C: ${A__B__C}
```

the result is

```
INFO - ABC: C
INFO - ABC__: C
INFO - A__B__C: C
```

Impact



Three **Robot Framework** variables with different names are created, but all variables have the same value. Therefore, underscores must not be used to distinguish between variables!

2.11 Version control interfaces

The **RobotFramework AIO** supports a version control in two ways: *implicitly* and *explicitly*. An implicit version control is executed automatically at the beginning of every test suite (during `testsuite_setup`). A version expected by user and defined within the variant configuration file, is checked against the version of the installed **RobotFramework AIO**. In detail this is explained in 2.2.2.

Version control can also be executed explicitly by the user. This version control can refer to any versions (not only to the versions of already installed software such as **RobotFramework AIO** and **TestsuitesManagement**).

Additionally, it is possible to query the versions of **RobotFramework AIO** and **TestsuitesManagement** directly. For this purpose, **TestsuitesManagement** provides two interfaces:

1. **RobotFramework AIO** interface
2. **Python** interface

2.11.1 RobotFramework AIO interface for version control

Like usual, the **TestsuitesManagement** library needs to be imported in this way:

```
Library      RobotFramework_TestsuitesManagement    WITH NAME    tm
```

2.11.2 Explicit version control of the RobotFramework AIO

The simplest implementation of a version control looks like this:

```

${result}=    tm.check_version    min_version=0.1.0    max_version=0.20.0
log          === check_version result: ${result}    console=yes

```

In this example, the specification is missing against which version (the `reference_version`) the `min_version` and the `max_version` should be checked. This automatically leads to the version of the installed software being taken as reference. There are two possibilities for this:

1. The **TestsuitesManagement** is installed together with the **RobotFramework AIO** (thus is part of the entire **RobotFramework AIO** bundle). In this case, the version of the installed **RobotFramework AIO** is the *reference* version.
2. The **TestsuitesManagement** is installed standalone (for example separately via PyPi). In this case, the version of the installed **TestsuitesManagement** is the *reference* version.

We assume now that the **RobotFramework AIO** is installed with version **0.14.2.26**. This fits to the expectation of minimum version **0.1.0** and maximum version **0.20.0** from the example above. Therefore the result is:

```

INFO - Version check passed (RobotFramework AIO (extended))
INFO - === check_version result: True

```

Exactly this is what already happened during the `testsuite_setup`. Therefore this version check will not provide any new insights. In the next example we check the version of the **TestsuitesManagement**.

2.11.3 Explicit version control of the TestsuitesManagement

At first we get the version of the **TestsuitesManagement**. After this we set this version as reference:

```
${tsm_version}=    tm.get_tsm_version
log      === tsm_version: ${tsm_version}    console=yes
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0    ↵
    ↪ reference_version=${tsm_version}
log      === check_version result (TSM): ${result}    console=yes
```

It can be seen that the expectation of minimum version **0.1.0** and maximum version **0.2.0** does not fit to the current **TestsuitesManagement** version **0.9.0**. The version check and therefore also the test fails:

```
ERROR - Version check failed: The test execution requires the maximum version '0.2.0', ↵
    ↪ but the reference version '0.9.0' is younger
UNKNOWN - Execution will be aborted because of a failed version check
```

Whether the test result is **UNKNOWN** or **FAIL** depends on the type of the **RobotFramework AIO** installation. Details can be found here: robotframework-aio.org.

2.11.4 User defined version control

The keyword **check_version** can be used for any version checks:

```
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0    ↵
    ↪ reference_version=1.2.3
```

Versions that are not relevant for a version check, can be set to **None**. In this example only the maximum version is checked against the reference:

```
${result}=    tm.check_version    min_version=${None}    max_version=0.2.0    ↵
    ↪ reference_version=1.2.3
```

If nothing is specified, the version check is skipped:

```
${result}=    tm.check_version    min_version=${None}    max_version=${None}    ↵
    ↪ reference_version=${None}
```

This is the same like:

```
${result}=    tm.check_version    min_version=${None}    max_version=${None}
```

Because of a version check is not executed, the reference version doesn't matter.

Result:

```
INFO - Version check is skipped because both 'min_version' and 'max_version' are set to ↵
    ↪ None (RobotFramework AIO (extended))
```

The following formats are accepted for all version parameters:

- **major.minor.patch**
- **major.minor**
- **major**

Therefore, this is also possible:

```
${result}=    tm.check_version    min_version=0    max_version=2    reference_version=3
```

With result:

```
ERROR - Version check failed: The test execution requires the maximum version '2', but ↵
    ↪ the reference version '3' is younger
UNKNOWN - Execution will be aborted because of a failed version check
```

Every deviation cause a failed test:

```
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0.9    ↵
               ↳ reference_version=1.2.3
```

Result:

```
ERROR - Version format error: maximum version '0.2.0.9' contains too many parts ↵
        ↳ (expected is: major.minor.patch)
UNKNOWN - Execution will be aborted because of a critical version check issue
```

It makes a difference, whether the reference version is defined or not!

- `reference_version` is not defined:

Depending on the installed software, the `TestsuitesManagement` decides what is used as reference (either the **RobotFramework AIO** or the **TestsuitesManagement**). This means, a name of the checked software is known. In this case the name is part of the logged output:

```
${result}=    tm.check_version    min_version=${None}    max_version=${None}
```

```
INFO - Version check is skipped because both 'min_version' and 'max_version' are set to ↵
        ↳ None (RobotFramework AIO (extended))
```

- `reference_version` is defined:

In this case a corresponding application name that is assigned to this number, is not available. Therefore, the log does not contain a name.

```
${result}=    tm.check_version    min_version=0.1.0    max_version=0.2.0    ↵
               ↳ reference_version=1.2.3
```

```
INFO - Version check is skipped because both 'min_version' and 'max_version' are set to None
```

This is the same result as before, but without name (**RobotFramework AIO (extended)**).

2.11.5 Python interface for version control

The goal of this Python interface is to enable **RobotFramework AIO** users who work purely in Python, without using robot files, to verify which bundle version is installed. The bundle version is either the version of the entire **RobotFramework AIO** or, if installed standalone, the version of the **TestsuitesManagement**.

Example

```
from RobotFramework_TestsuitesManagement.Utils.CVersion import CVersion
oVersion = CVersion()
result, reason = oVersion.verifyVersion(min_version, max_version)
```

The parameters `min_version` and `max_version` define the expected range of the bundle version. This is verified against the version of the currently installed bundle. The format of the version string is: `\d\.\d\.\d`.

It is possible to set a version to `None` or to `""`. The impact is that this version number (*min* or *max*) is not part of the version check.

Two values are returned:

1. `result`
A boolean variable that is either `True` in case of the version check is passed or `False` in case of the version check fails.
2. `reason`
In case of the version check is passed, `reason` is `None`. In case the version check fails, `reason` contains a short token string indicating the reason for the failure.

The method `verifyVersion` does not return a full error message, but a small token string only. This enables users to formulate their own error messages.

These are the token strings together with their meanings:

- `without_version_check`
Version check not executed.
- `wrong_minmax`
Mismatch of *minimum version* and *maximum version*: The *minimum version* is younger than the *maximum version*.
- `conflict_min`
The test execution requires the *minimum version*, but the installed version is older.
- `conflict_max`
The test execution requires the *maximum version*, but the installed version is younger.
- `internal_error`
An internal error happened (any issue in JSON configuration file).

Chapter 3

The JSONP format

The JSONP format is computed by the Python application **JsonPreprocessor**. Additionally to this, it is also possible to use the JSONP format within tests of a test automation framework called **RobotFramework AIO** ([homepage](#)).

The component that within the **RobotFramework AIO** is responsible for the execution of the **JsonPreprocessor**, is called **TestsuitesManagement**. Because the **JsonPreprocessor** is a pure Python application, whereas the **RobotFramework AIO** is a framework with own syntax rules and conditions, in some cases the JSONP format deviates in both worlds. Where required you will find corresponding hints in this chapter. In all other cases the JSONP format is the same in **JsonPreprocessor** and **RobotFramework AIO**.

This chapter explains the format of JSONP files in detail. We concentrate here on the content of the JSONP files and the corresponding results, available in Python dictionary format.

3.1 Standard JSON format

The **JsonPreprocessor** supports JSON files with standard extension `.json` and standard content.

• JSON file:

```
{
  "param1" : "value1",
  "param2" : "value2"
}
```

Outcome:

```
{'param1': 'value1', 'param2': 'value2'}
```

A JSON file with extension `.jsonp` and same content will produce the same output.

We recommend to give every JSON file the extension `.jsonp` to have a strict separation between the standard and the extended JSON format.

The following example still contains standard JSON content, but with parameters of several different data types (simple and composite).

```
{
  "param_01" : "string",
  "param_02" : 123,
  "param_03" : 4.56,
  "param_04" : ["A", "B", "C"],
  "param_05" : {"A" : 1, "B" : 2, "C" : 3}
}
```

This content produces the following output:

```
{'param_01': 'string',  
'param_02': 123,  
'param_03': 4.56,  
'param_04': ['A', 'B', 'C'],  
'param_05': {'A': 1, 'B': 2, 'C': 3}}
```

This output is of a certain dictionary type (named *dotdict*) that allows to access elements also with an object oriented dot notation (details about this format can be found in section [dotdict notation](#)).

3.2 Boolean and null values

JSON supports the boolean values `true` and `false`, and also the null value `null`.

In Python, the corresponding values differ: `True`, `False` and `None`.

Because the **JsonPreprocessor** is a Python application and therefore the returned content is required to be formatted Python compatible, the **JsonPreprocessor** does a conversion automatically.

Accepted in JSON files are both styles:

```
{
  "param_06" : true,
  "param_07" : false,
  "param_08" : null,
  "param_09" : True,
  "param_10" : False,
  "param_11" : None
}
```

The output contains all keywords in Python style only:

```
{'param_06': True,
 'param_07': False,
 'param_08': None,
 'param_09': True,
 'param_10': False,
 'param_11': None}
```

3.3 Comments

Comments can be added to JSON files with `//` :

```
{
  // JSON keywords
  "param_06" : true,
  "param_07" : false,
  "param_08" : null,
  // Python keywords
  "param_09" : True,
  "param_10" : False,
  "param_11" : None
}
```

All lines starting with `//` are ignored by the **JsonPreprocessor**. The output of this example is the same as in the previous example.

Also block comments and inline comments are possible, realized by a pair of `/* */` :

```
{
  /*
  "param1" : 1,
  "param2" : "A",
  */

  "testlist" : ["A1", /*"B2", "C3",*/ "D4"]
}
```

Outcome:

```
{'testlist': ['A1', 'D4']}
```

3.4 Import of JSON files

We assume the following scenario:

A software component *A* requires a set of configuration parameters. A software component *B* that belongs to the same main software or to the same project, requires another set of configuration parameters. Additionally both components require a common set of parameters (with the same values).

The outcome is that at least we need two JSON configuration files:

1. A file `componentA.jsonp` containing all parameters required for component *A*
2. A file `componentB.jsonp` containing all parameters required for component *B*

But with this solution both JSON files would contain also the common set of parameters. This is unfavorable, because the corresponding values need to be maintained at two different positions.

Therefore we extend the list of JSON files by a file containing the common part only:

1. A file `common.jsonp` containing all parameters that are the same for component *A* and component *B*
2. A file `componentA.jsonp` containing remaining parameters (with specific values) required for component *A*
3. A file `componentB.jsonp` containing remaining parameters (with specific values) required for component *B*

Finally we use the import mechanism of the **JsonPreprocessor** to import the file `common.jsonp` in file `componentA.jsonp` and also in file `componentB.jsonp`.

This can be the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : "componentA value 1",
  "componentA_param_2" : "componentA value 2"
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2"
}
```

Explanation:

JSON files are imported with the key `"[import]"`. The value of this key is the path and name of the JSON file to be imported.

A JSON file can contain more than one import. Imports can be nested: An imported JSON file can import further JSON files also.

Outcome:

The file `componentA.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common value 2',
'componentA_param_1': 'componentA value 1',
'componentA_param_2': 'componentA value 2'}
```

The file `componentB.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

It can be seen that the returned dictionary contains both the parameters from the loaded JSON file and the parameters imported by the loaded JSON file.

Multiple imports of the same file

A JSONP file can be imported more than once anywhere in the set of JSONP configuration files. This mechanism can be used to define a common part for different subkeys.

Example:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // component A parameters
  "componentA_param_1" : {
    "componentA_param_1_a" : "componentA_param_1_a value",
    // common parameters within componentA_param_1
    "[import]" : "../common_config/common.jsonp",
    "componentA_param_1_b" : "componentA_param_1_b value"
  },
  "componentA_param_2" : {
    "componentA_param_2_a" : "componentA_param_2_a value",
    // common parameters within componentA_param_2
    "[import]" : "../common_config/common.jsonp",
    "componentA_param_2_b" : "componentA_param_2_b value"
  }
}
```

Outcome:

The content of `common.jsonp` is part of `componentA_param_1` and also part of `componentA_param_2`.

```
{'componentA_param_1': {'common_param_1': 'common value 1',
                        'common_param_2': 'common value 2',
                        'componentA_param_1_a': 'componentA_param_1_a value',
                        'componentA_param_1_b': 'componentA_param_1_b value'},
'componentA_param_2': {'common_param_1': 'common value 1',
                        'common_param_2': 'common value 2',
                        'componentA_param_2_a': 'componentA_param_2_a value',
                        'componentA_param_2_b': 'componentA_param_2_b value'}}
```

Dynamic import paths

The values of `"[import]"` keys must be of type `str`. Therefore it's possible to use dollar operator expressions inside import paths.

An alternative version of the file `componentA.jsonp` from the example above, can look like this:

```
{
  "common_config_dir" : "./common_config",
  // component A parameters
  "componentA_param_1" : {
    "componentA_param_1_a" : "componentA_param_1_a value",
    // common parameters within componentA_param_1
    "[import]" : "${common_config_dir}/common.jsonp",
    "componentA_param_1_b" : "componentA_param_1_b value"
  },
  "componentA_param_2" : {
    "componentA_param_2_a" : "componentA_param_2_a value",
    // common parameters within componentA_param_2
    "[import]" : "${common_config_dir}/common.jsonp",
    "componentA_param_2_b" : "componentA_param_2_b value"
  }
}
```

The outcome is the same like in the previous example.

3.5 Overwriting parameters

We take over the scenario from the previous section: We still have a JSON file `componentA.jsonp` containing the parameters for component *A*, a JSON file `componentB.jsonp` for component *B* and a JSON file `common.jsonp` for both components.

But now component *B* requires a different value of a common parameter: Within a JSON file we need to change the value of a parameter that is initialized within an imported file. That is possible.

This is now the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : "common value 1",
  "common_param_2" : "common value 2"
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : "componentA value 1",
  "componentA_param_2" : "componentA value 2"
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "../common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2",
  // overwrite parameter initialized by imported file
  "common_param_2" : "common componentB value 2"
}
```

Explanation:

With

```
"common_param_2" : "common componentB value 2"
```

in `componentB.jsonp`, the initial definition

```
"common_param_2" : "common value 2"
```

in `common.jsonp` is overwritten.

Outcome:

The file `componentB.jsonp` produces the following output:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common componentB value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

Important: *The value a parameter has finally, depends on the order of definitions, redefinitions and imports!*

In file `componentB.jsonp` we move the import of `common.jsonp` to the bottom:

```
{
  // component B parameters
  "componentB_param_1" : "componentB value 1",
  "componentB_param_2" : "componentB value 2",
  "common_param_2" : "common componentB value 2"
  //
  // common parameters
  "[import]" : "./common.jsonp",
}
```

Now the imported file overwrites the value initialized in the importing file.

Outcome:

```
{'common_param_1': 'common value 1',
'common_param_2': 'common value 2',
'componentB_param_1': 'componentB value 1',
'componentB_param_2': 'componentB value 2'}
```

Up to now we considered simple data types only. In case we want to overwrite a parameter that is part of a composite data type, we need to extend the syntax. This is explained in the next examples.

Again we take over the scenario from the previous section: We still have a JSON file `componentA.jsonp` containing the parameters for component *A*, a JSON file `componentB.jsonp` for component *B* and a JSON file `common.jsonp` for both components.

But now all values are part of composite data types like lists and dictionaries.

This is the content of the JSON files:

- `common.jsonp`

```
{
  // common parameters
  "common_param_1" : ["common value 1.1", "common value 1.2"],
  "common_param_2" : {"common_key_2_1" : "common value 2.1",
                      "common_key_2_2" : "common value 2.2"}
}
```

- `componentA.jsonp`

```
{
  // common parameters
  "[import]" : "./common.jsonp",
  //
  // component A parameters
  "componentA_param_1" : ["componentA value 1.1", "componentA value 1.2"],
  "componentA_param_2" : {"componentA_key_2_1" : "componentA value 2.1" ,
                          "componentA_key_2_2" : "componentA value 2.2"}
}
```

- `componentB.jsonp`

```
{
  // common parameters
  "[import]" : "./common.jsonp",
  //
  // component B parameters
  "componentB_param_1" : ["componentB value 1.1", "componentB value 1.2"],
  "componentB_param_2" : {"componentB_key_2_1" : "componentB value 2.1" ,
                          "componentB_key_2_2" : "componentB value 2.2"}
}
```

Like in previous examples, the outcome is a merge of the imported JSON file and the importing JSON file, e.g. for `componentA.jsonp` :

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentA_param_1': ['componentA value 1.1', 'componentA value 1.2'],
 'componentA_param_2': {'componentA_key_2_1': 'componentA value 2.1',
                        'componentA_key_2_2': 'componentA value 2.2'}}
```

Now the following questions need to be answered:

1. How to get the value of an already existing parameter?
2. How to get the value of a single element of a parameter of nested data type (list, dictionary)?
3. How to overwrite the value of a single element of a parameter of nested data type?
4. How to add an element to a parameter of nested data type?

We introduce another JSON file `componentB.2.jsonp` in which we import the JSON file `componentB.jsonp` . In this file we also add content to work with simple and composite data types to answer the questions above.

We introduce a new file `componentB.2.jsonp` that imports `componentB.jsonp` and creates new parameters based on already existing parameters:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  // some additional parameters of simple data type
  "string_val" : "ABC",
  "int_val"    : 123,
  "float_val"  : 4.56,
  "bool_val"   : true,
  "null_val"   : null,

  // access to existing parameters
  "string_val_b"      : ${string_val},
  "int_val_b"         : ${int_val},
  "float_val_b"       : ${float_val},
  "bool_val_b"        : ${bool_val},
  "null_val_b"        : ${null_val},
  "common_param_1_b"  : ${common_param_1},
  "componentB_param_2_b" : ${componentB_param_2}
}
```

Outcome:

```
{'bool_val': True,
 'bool_val_b': True,
 'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_1_b': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'},
 'componentB_param_2_b': {'componentB_key_2_1': 'componentB value 2.1',
                           'componentB_key_2_2': 'componentB value 2.2'},
 'float_val': 4.56,
 'float_val_b': 4.56,
 'int_val': 123,
 'int_val_b': 123,
 'null_val': None,
 'null_val_b': None,
 'string_val': 'ABC',
 'string_val_b': 'ABC'}
```

The rules for accessing parameters are:

- Existing parameters are accessed by a dollar operator and a pair of curly brackets (`${...}`) with the parameter name inside.
- If the entire expression of the right-hand side of the colon is such a dollar operator expression, it is not required to encapsulate this expression in quotes (in opposite to what pure JSON would require).
- Without quotes, the dollar operator keeps the data type of the referenced parameter. If you use quotes, the value of the used parameter will be converted to type `str`. This implicit string conversion is limited to parameters of simple data types like integers and floats. Composite data types like lists and dictionaries cannot be used for that.

In more detail:

The dollar operator keeps the data type of the referenced parameter. In case of `int_val` is of type `int`, also `int_val_b` is of type `int`:

```
"int_val_b" : ${int_val},
```

It is not required to encapsulate dollar operator expressions at the right-hand side of the colon in quotes. But nevertheless, it is possible to use quotes. In case of:

```
"int_val_b" : "${int_val}",
```

the parameter `int_val_b` is of type `str`.

Further content can be added between the double quotes. This can be used to create composite strings:

```
"str_val" : "ABC",
"int_val" : 1,
"float_val" : 2.3,
"bool_val" : True,
"none_val" : None,
"list_val" : [1,2,3],
"dict_val" : {"A" : "B"},
"newparam1" : "prefix_${str_val}_suffix",
"newparam2" : "prefix_${int_val}_suffix",
"newparam3" : "prefix_${float_val}_suffix",
"newparam4" : "prefix_${bool_val}_suffix",
"newparam5" : "prefix_${none_val}_suffix"
```

Outcome:

```
{'bool_val': True,
 'dict_val': {'A': 'B'},
 'float_val': 2.3,
 'int_val': 1,
 'list_val': [1, 2, 3],
 'newparam1': 'prefix_ABC_suffix',
 'newparam2': 'prefix_1_suffix',
 'newparam3': 'prefix_2.3_suffix',
 'newparam4': 'prefix_True_suffix',
 'newparam5': 'prefix_None_suffix',
 'none_val': None,
 'str_val': 'ABC'}
```

Using composite data types inside strings is not supported:

```
"newparam6" : "prefix_${listval}_suffix"
```

or:

```
"newparam7" : "prefix_${dictval}_suffix"
```

Result for `"newparam6"`:

```
The substitution of parameter '${listval}' inside the string value ↵
↵ 'prefix_${listval}_suffix' is not supported! Composite data types like lists and ↵
↵ dictionaries cannot be substituted inside strings.
```

Value of a single element of a parameter of nested data type

To access an element of a list and a key of a dictionary, we change the content of file `componentB.2.jsonp` to:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  "list_element_0" : ${componentB_param_1}[0],
  "dict_key_2_2"    : ${common_param_2}['common_key_2_2']
}
```

Outcome:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'},
 'dict_key_2_2': 'common value 2.2',
 'list_element_0': 'componentB value 1.1'}
```

Overwrite the value of a single element of a parameter of nested data type

In the next example we overwrite the value of a list element and the value of a dictionary key.

Again we change the content of file `componentB.2.jsonp` :

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  ${componentB_param_1}[0]      : "componentB value 1.1 (new)",
  ${common_param_2}['common_key_2_1'] : "common value 2.1 (new)"
}
```

The dollar operator syntax at the left-hand side of the colon is the same as previously used on the right-hand side. The entire expression at the left-hand side of the colon must *not* be encapsulated in quotes in this case.

Outcome:

The single elements of the list and the dictionary are updated, all other elements are unchanged.

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1 (new)',
                    'common_key_2_2': 'common value 2.2'},
 'componentB_param_1': ['componentB value 1.1 (new)', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'}}
```

Add an element to a parameter of nested data type

Adding further elements to an already existing list is not possible in JSON! But it is possible to add keys to an already existing dictionary.

The following example extends the dictionary `common_param_2` by an additional key `common_key_2_3`:

```
{
  // import of componentB parameters
  "[import]" : "./componentB.jsonp",
  //
  ${common_param_2}["common_key_2_3"] : "common value 2.3"
}
```

Outcome:

```
{'common_param_1': ['common value 1.1', 'common value 1.2'],
 'common_param_2': {'common_key_2_1': 'common value 2.1',
                    'common_key_2_2': 'common value 2.2',
                    'common_key_2_3': 'common value 2.3'},
 'componentB_param_1': ['componentB value 1.1', 'componentB value 1.2'],
 'componentB_param_2': {'componentB_key_2_1': 'componentB value 2.1',
                        'componentB_key_2_2': 'componentB value 2.2'}}
```

Dictionary keys and indices as parameter

In all code examples above the indices of lists and the key names of dictionaries have been hard coded strings. It is also possible to use parameters:

```
{
  "index1"      : 0,
  "index2"      : 1,
  "key1"        : "keyA",
  "key2"        : "keyB",
  "testlist"    : ["A", "B"],
  "testdict"    : {"keyA" : "A", "keyB" : "B"},
  "tmp1"        : ${testlist}[${index1}],
  "tmp2"        : ${testdict}[${key1}],
  ${testlist}[${index1}] : ${testlist}[${index2}],
  ${testdict}[${key1}]   : ${testdict}[${key2}],
  ${testlist}[${index2}] : ${tmp1},
  ${testdict}[${key2}]   : ${tmp2}
}
```

Outcome:

```
{'index1': 0,
 'index2': 1,
 'key1': 'keyA',
 'key2': 'keyB',
 'testdict': {'keyA': 'B', 'keyB': 'A'},
 'testlist': ['B', 'A'],
 'tmp1': 'A',
 'tmp2': 'A'}
```

Meaning of single quotes in square brackets

Single quotes are used to convert the content inside to a string.

- In case of the parameter `param` is of type `str`, the expressions `[$ {param}]` and `[' ${param} ']` have the same outcome: The content inside the square brackets is a string. The single quotes have no meaning in this case (because the parameter is already of type `str`).
- In case of the parameter `param` is of type integer, the quotes in `[' ${param} ']` convert the integer value to a string. Without the quotes (`[$ {param}]`), the content inside the square brackets is an integer.

In the context of **JsonPreprocessor** JSON files, only strings and integers are expected to be inside square brackets (except the brackets are used to define a list). Other data types are not supported here.

Whether a string or an integer is expected, depends on the data type of the parameter, the square bracket expression belongs to. Dictionaries require a string (a key name), lists require an integer (an index). Deviations will cause an error.

Summarized the following combinations are valid (on both the left-hand side of the colon and the right-hand side of the colon):

```

${listparam}[$ {intparam}]
${listparam}[1]
${dictparam}[' ${intparam} ']
${dictparam}[$ {stringparam}]
${dictparam}[' ${stringparam} ']
${dictparam}['keyname']

```

Use of a common dictionary

The last example in this section covers the following use case:

- We have several JSON files, each for a certain purpose within a project (e.g. for every feature of this project a separate JSON file).
- They belong together and therefore they are all imported into a main JSON file that is the file that is handed over to the **JsonPreprocessor**.
- Every imported JSON file introduces a certain bunch of parameters. All parameters need to be a part of a common dictionary.
- Outcome is that finally only one single dictionary is used to access the parameters from all JSON files imported in the main JSON file.

These are the JSON files:

- `project.jsonp`

```
{
  // define some common values
  ${project_values}['common_project_param_1'] : "common project value 1",
  ${project_values}['common_project_param_2'] : "common project value 2",
  //
  // import feature parameters
  "[import]" : "./featureA.jsonp",
  "[import]" : "./featureB.jsonp",
  "[import]" : "./featureC.jsonp"
}
```

- `featureA.jsonp`

```
{
  // parameters required for feature A
  ${project_values}['featureA_params']['featureA_param_1'] : "featureA param 1 value",
  ${project_values}['featureA_params']['featureA_param_2'] : "featureA param 2 value"
}
```

- `featureB.jsonp`

```
{
  // parameters required for feature B
  ${project_values}['featureB_params']['featureB_param_1'] : "featureB param 1 value",
  ${project_values}['featureB_params']['featureB_param_2'] : "featureB param 2 value"
}
```

- `featureC.jsonp`

```
{
  // parameters required for feature C
  ${project_values}['featureC_params']['featureC_param_1'] : "featureC param 1 value",
  ${project_values}['featureC_params']['featureC_param_2'] : "featureC param 2 value"
}
```

It is not required to start the code listed above, with dictionary initializations like

```
"project_values" : {},
${project_values}['featureA_params'] : {},
${project_values}['featureB_params'] : {},
${project_values}['featureC_params'] : {},
```

These initializations are done implicitly by the **JsonPreprocessor**. Further details about the implicit creation of dictionaries can be found in section [Implicit creation of dictionaries](#).

It is for sure still possible to do the initialization of a dictionary explicitly with `{}`. But keep in mind: This deletes all already existing keys in this dictionary!

Outcome:

```
{'project_values': {'common_project_param_1': 'common project value 1',
                    'common_project_param_2': 'common project value 2',
                    'featureA_params': {'featureA_param_1': 'featureA param 1 value',
                                         'featureA_param_2': 'featureA param 2 value'},
                    'featureB_params': {'featureB_param_1': 'featureB param 1 value',
                                         'featureB_param_2': 'featureB param 2 value'},
                    'featureC_params': {'featureC_param_1': 'featureC param 1 value',
                                         'featureC_param_2': 'featureC param 2 value'}}
```

3.6 dotdict notation

Up to now we have accessed dictionary keys in this way (standard notation):

```
${dictionary}['key']['sub_key']
```

Additionally to this standard notation, the **JsonPreprocessor** supports the so called *dotdict* notation where keys are handled as attributes:

```
${dictionary.key.sub_key}
```

In standard notation keys are encapsulated in square brackets and all together is placed *outside* the curly brackets. In dotdict notation the dictionary name and the keys are separated by dots from each other. All together is placed *inside* the curly brackets.

In standard notation key names are allowed to contain dots:

```
${dictionary}['key']['sub.key']
```

In dotdict notation this would cause ambiguities:

```
${dictionary.key.sub.key}
```

Therefore it is not possible to implement in this way! In case you need to have dots inside key names, you must use the standard notation. We recommend to prefer underlines as separator - like done in the examples in this document.

Do you really need dots inside key names?

Please keep in mind: The dotdict notation is a reduced one. Because of parts are missing (e.g. the single quotes around key names), the outcome can be code that is really hard to capture.

In the following example we create a composite data structure and demonstrate how to access single elements in both notations.

- JSON file:

```
{
  // composite data structure
  "params" : [{"dict_1_key_1" : "dict_1_key_1 value",
    "dict_1_key_2" : ["dict_1_key_2 value 1", "dict_1_key_2 value 2"]},
    //
    {"dict_2_key_1" : "dict_2_key_1 value",
    "dict_2_key_2" : {"dict_2_A_key_1" : "dict_2_A_key_1 value",
      "dict_2_A_key_2" : ["dict_2_A_key_2 value 1", ↵
↵ "dict_2_A_key_2 value 2"]}}}],
  //
  // access to single elements of composite data structure
  //
  // a) standard notation
  "dict_1_key_2_value_2_standard" : ${params}[0]['dict_1_key_2'][1],
  // b) dotdict notation
  "dict_1_key_2_value_2_dotdict" : ${params.0.dict_1_key_2.1},
  //
  // c) standard notation
  "dict_2_A_key_2_value_2_standard" : ${params}[1]['dict_2_key_2']['dict_2_A_key_2'][1]
  // d) dotdict notation
  "dict_2_A_key_2_value_2_dotdict" : ${params.1.dict_2_key_2.dict_2_A_key_2.1}
}
```

Outcome:

In case of the composite data structure becomes more and more nested (and if also the key names contain numbers), understanding the expressions (like `${params.1.dict_2_key_2.dict_2_A_key_2.1}`) becomes more and more challenging!

```
{'dict_1_key_2_value_2_standard': 'dict_1_key_2 value 2',
 'dict_2_A_key_2_value_2_standard': 'dict_2_A_key_2 value 2',
 'params': [{ 'dict_1_key_1': 'dict_1_key_1 value',
               'dict_1_key_2': ['dict_1_key_2 value 1', 'dict_1_key_2 value 2']},
             { 'dict_2_key_1': 'dict_2_key_1 value',
               'dict_2_key_2': { 'dict_2_A_key_1': 'dict_2_A_key_1 value',
                                'dict_2_A_key_2': ['dict_2_A_key_2 value 1',
                                                    'dict_2_A_key_2 value 2']}}]}
```

3.7 Dynamic key names

In section [Overwriting parameters](#) we mentioned the possibility to define the value of string parameters dynamically, e.g. in this way:

```
"str_val" : "ABC",
"newparam1" : "prefix_${str_val}_suffix",
```

The value of `newparam1` is defined by an expression that is encapsulated in quotes and contains - beneath hard coded parts - a dollar operator expression (that is the dynamic part).

The same is also possible on the left-hand side of the colon. In this case the name of a parameter is created dynamically.

Example:

```
"strval" : "A",
"dictval" : {"A_2" : 1},
${dictval}['${strval}_2'] : 2
```

In second line a new dictionary with key `A_2` is defined. In third line we overwrite the initial value of this key with another value. The name of this key is defined with the help of parameter `strval`.

Outcome:

```
{'dictval': {'A_2': 2}, 'strval': 'A'}
```

The same in dotdict notation:

```
"strval" : "A",
"dictval" : {"A_2" : 1},
${dictval}.${strval}_2 : 3
```

The precondition for using dynamic key names is that a key with the resulting name (here `A_2`) does exist already. Therefore this mechanism can be used to overwrite the value of existing keys, but cannot be used to create new keys!

This will not work (because of a key with name `A_2` does not yet exist):

```
"strval" : "A",
"dictval" : {"${strval}_2" : 1}
```

Outcome:

```
A substitution in key names is not allowed! Please update the key name "${strval}_2"
```

3.8 Implicit creation of dictionaries

Up to now we have discussed two different ways of creating nested dictionaries.

The first one is “on the fly”, like:

```
{
  "project_values" : {"keyA" : "keyA value",
                     "keyB" : {"keyB1" : "keyB1 value",
                              "keyB2" : {"keyB21" : "keyB21 value",
                                         "keyB22" : "keyB22 value"}}}}
}
```

In case of it is required to split the definition into several files, we have to add keys (and also the initialization) line by line:

```
{
  "project_values" : {},
  ${project_values}['keyA'] : "keyA value",
  ${project_values}['keyB'] : {},
  ${project_values}['keyB']['keyB1'] : "keyB1 value",
  ${project_values}['keyB']['keyB2'] : {},
  ${project_values}['keyB']['keyB2']['keyB21'] : "keyB21 value",
  ${project_values}['keyB']['keyB2']['keyB22'] : "keyB22 value"
}
```

The result will be the same as in the previous example.

It can be seen now that this way of creating nested dictionaries is rather long-winded, because every initialization of a dictionary requires a separate line of code (at every level).

To shorten the code, the **JsonPreprocessor** supports an implicit creation of dictionaries.

This is the resulting code in standard notation:

```
{
  ${project_values}['keyA'] : "keyA value",
  ${project_values}['keyB']['keyB1'] : "keyB1 value",
  ${project_values}['keyB']['keyB2']['keyB21'] : "keyB21 value",
  ${project_values}['keyB']['keyB2']['keyB22'] : "keyB22 value"
}
```

And the same in dotdict notation (with precondition, that no key name contains a dot):

```
{
  ${project_values.keyA} : "keyA value",
  ${project_values.keyB.keyB1} : "keyB1 value",
  ${project_values.keyB.keyB2.keyB21} : "keyB21 value",
  ${project_values.keyB.keyB2.keyB22} : "keyB22 value"
}
```

Caution:

We urgently recommend *not* to mixup both styles in one line of code. In case of keys contain a list and also numerical indices are involved, we recommend to prefer the standard notation.

Please be aware of: In case of a missing level in between an expression like

```
{
  ${project_values.keyB.keyB2.keyB22} : "keyB22 value"
}
```

you will *not* get an error message! The entire data structure will be created implicitly. The impact is that this method is very susceptible to typing mistakes.

The implicit creation of data structures does not work with lists! In case you use a list index out of range, you will get a corresponding error message.

Key names

The implicit creation of data structures is only possible with *hard coded* key names. Parameters are not supported.

Example:

```
{
  "paramA" : "ABC",
  "subKey" : "ABC",
  ${testdict.subKey.subKey.paramA} : "DEF"
}
```

All sub key levels within the expression `${testdict.subKey.subKey.paramA}` are interpreted as hard coded strings, even in case of parameters with the same name do exist.

For example: The name of the implicitly created key at bottom level is `"paramA"`, and not the value `"ABC"` of the parameter with the same name (`"paramA"`).

Therefore the outcome is:

```
{'paramA': 'ABC', 'subKey': 'ABC', 'testdict': {'subKey': {'subKey': {'paramA': 'DEF'}}}}
```

Reference to existing keys

It is possible to use parameters to refer to *already existing* keys.

```
{
  // data structure created implicitly
  ${testdict.subKey_1.subKey_2.subKey_3} : "ABC",

  // string parameter with name of an existing key
  "keyName_3" : "subKey_3",

  // parameter used to refer to an existing key
  ${testdict.subKey_1.subKey_2.${keyName_3}} : "XYZ"
}
```

Outcome:

```
{'keyName_3': 'subKey_3',
 'testdict': {'subKey_1': {'subKey_2': {'subKey_3': 'XYZ'}}}}
```

Parameters cannot be used to create new keys.

```
{
  // data structure created implicitly
  ${testdict.subKey_1.subKey_2.subKey_3} : "ABC",

  // string parameter with name of a not existing key
  "keyName_4" : "subKey_4",

  // usage of keyName_4 is not possible here
  ${testdict.subKey_1.subKey_2.subKey_3.${keyName_4}} : "XYZ"
}
```

Outcome is the following error:

```
"The implicit creation of data structures based on nested parameter is not supported ..."
```

The same error will happen in case of the standard notation is used:

```
{
  // usage of keyName_4 is not possible here
  ${testdict}['subKey_1']['subKey_2']['subKey_3'][${keyName_4}] : "XYZ"
}
```

3.9 Python inline code

It might be required to have access to Python builtin functions, e.g. for combining multiple lists into a single list. In pure Python this can be realized in the following way:

```
A = [1, 2, 3]
B = [4, 5, 6]
C = A + B
print(C)
```

With result:

```
[1, 2, 3, 4, 5, 6]
```

In JSONP files, the implementation looks as follows:

```
"A" : [1, 2, 3],
"B" : [4, 5, 6],
"C" : <<${A} + ${B}>>
```

The expression `${A} + ${B}` is handled as Python inline code. It is possible to use dollar operator expressions inside Python inline code to access any other parameter defined in JSONP files.

The syntax of Python inline code is:

```
<<(Python expression)>>
```

Limitations

The Python inline code syntax cannot be used within strings. The following code is invalid:

```
"param" : "The value is <<(Python expression)>>"
```

The usage of Python inline code is limited to the right-hand side of the colon (*key values*). This feature cannot be used to create key names.

Python inline code cannot be used immediately to select list elements or dictionary keys that shall be overwritten. The following code is invalid:

```
"param" : [1, 2, 3],
${param}[<<0 if True else 1>>] : 4
```

This is a possible workaround:

```
"param" : [1, 2, 3],
"index" : <<0 if True else 1>>,
${param}[${index}] : 4
```

Further examples

Create a list of dictionary keys:

```
"dict"      : {"kA" : "vA", "kB" : "vB"},
"key_list"  : <<list(${dict}.keys())>>
```

Define an import path:

```
"path_1" : "./file_1.jsonp",
"path_2" : "./file_2.jsonp"
//
"[import]" : <<${path_1} if True else ${path_2}>>
```

3.10 Special characters within key names

Basically, the **JsonPreprocessor** considers the JSON naming convention for key names. Within JSONP files allowed is what JSON allows, but with the following limitation: The JSONP format extends the JSON format with some features using square, curly and angle brackets as syntax elements: `[`, `]`, `{`, `}`, `<`, `>`.

These brackets must not be used within key names!

This JSONP code:

```
{
  "[" : 1,
  "B" : ${[]}
}
```

causes:

```
Error: 'Invalid expression found: '${[]}' - The brackets mismatch!!!'
```

All other special characters can be used immediately:

```
{
  "$" : 1,
  "B" : ${$}
}
```

Result:

```
DotDict({'$': 1, 'B': 1})
```

Strings are handled as raw strings. Therefore, masking has no effect:

```
{
  "\\$" : 1,
  "B" : ${\\$}
}
```

The backslashes are part of the key name:

```
DotDict({'\\$': 1, 'B': 1})
```

The masking requires two backslashes! A single backslash will cause a JSON syntax error!



Format deviation

In opposite to the **JsonPreprocessor**, the **TestsuitesManagement** of the **RobotFramework AIO** is more restrictive and does not allow any special characters. Key names are limited to letters and digits!

3.11 Byte sequences

The **TestsuitesManagement** configuration (in JSONP format) supports the explicit embedding of binary values (in Hexadecimal Escape Notation).

```
"byteSequence" : b'\x52\x6f\x62\x6f\x74\x46\x72\x61\x6d\x65\x77\x6f\x72\x6b\x20\x41\x49\x4f'
```

The result is the resolved content:

```
log    byteSequence: ${byteSequence}
```

```
INFO - byteSequence: RobotFramework AIO
```

This feature requires the usage of single quotes: `b'...'`. Double quotes must not be used.

Chapter 4

CConfig.py

4.1 Class: CConfig

Imported by:

```
from RobotFramework.TestsuitesManagement.Config.CConfig import CConfig
```

Defines the properties of configuration and holds the identified config files.

The loading configuration method is divided into 4 levels: level1 has the highest priority, level4 has the lowest priority.

Level1: Defined in command line

Level2: Read from content of JSON config file

```
{
  "default": {
    "name": "robot_config.jsonp",
    "path": "./config/"
  },
  "variant_0": {
    "name": "robot_config.jsonp",
    "path": "./config/"
  },
  "variant_1": {
    "name": "robot_config-variant_1.jsonp",
    "path": "./config/"
  },
  ...
  ...
}
```

According to the ConfigName, RobotFrameworkTestsuitesManagement will choose the corresponding config file. "./config/" indicates the relative path to json config file.

Level3: Read in testsuite folder: /config/robot_config.jsonp

Level4: Read from RobotFramework AIO installation folder:

```
/RobotFramework/defaultconfig/robot_config.jsonp
```

4.1.1 Method: loadCfg

This loadCfg method uses to load configuration's parameters from json files.

Arguments:

- No input parameter is required

Returns:

- No return variable

4.1.2 Method: checkVersion

This method validates the current package version with maximum and minimum version.

In case the current version is not between min and max version, then the execution of testsuite is terminated with "unknown" state

Chapter 5

CKeywords.py

5.1 Class: CSetupKeywords

Imported by:

```
from RobotFramework.TestsuitesManagement.Keywords.CKeywords import CSetupKeywords
```

This class defines the keywords for the setup and the teardown of testcases and testsuites.

5.1.1 Keyword: testsuite_setup

This keyword loads the RobotFramework AIO configuration, checks the version of the RobotFramework AIO and logs out the basic information about the test execution.

Arguments:

- `sTestsuiteCfgFile`
/ *Condition*: required / *Type*: string /
`sTestsuiteCfgFile=''` and variable `config_file` is not set RobotFramework AIO will check for configuration level 3, and level 4.
`sTestsuiteCfgFile` is set with a `<jsonconfig_file_path>` and variable `config_file` is not set RobotFramework AIO will load configuration level 2.

Returns:

- No return variable

5.1.2 Keyword: testsuite_teardown

This keyword writes information about the testsuite result to the log files.

5.1.3 Keyword: testcase_setup

This keyword writes the number of counted tests to the log files.

5.1.4 Keyword: testcase_teardown

This keyword writes information about the testcase result to the log files.

5.2 Class: CGeneralKeywords

Imported by:

```
from RobotFramework.TestsuitesManagement.Keywords.CKeywords import CGeneralKeywords
```

Class to define general keywords, that have nothing to do with the setups and teardowns of suites and tests.

5.2.1 Keyword: get_config

This getconfig defines the Get Config keyword gets the current config object of RobotFramework AIO.

Arguments:

- No parameter is required

Returns:

- oConfig.oConfigParams
/ *Type: json* /

5.2.2 Keyword: load_json

Loads a json file and returns a json object.

Arguments:

- jsonfile
/ *Condition: required* / *Type: string* /
The path of Json configuration file.
- level
/ *Condition: required* / *Type: int* /
Level = 1 -> loads the content of jsonfile.
level != 1 -> loads the json file which is set with variant (likes loading config level2)

Returns:

- oJsonData
/ *Type: json* /

5.2.3 Keyword: is_robotframework_aio

Returns * True: RobotFramework AIO is installed * False: RobotFramework AIO is not installed (= standalone installation of TestsuitesManagement)

5.2.4 Keyword: get_package_context_file

Returns path and name of package context file

5.2.5 Keyword: get_reference_version

Returns the version number used as reference for version checks. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

5.2.6 Keyword: get_reference_version_date

Returns the version date belonging to the reference. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

5.2.7 Keyword: `get_reference_app_name`

Returns the name of the reference application. The reference is either the RobotFramework AIO or the Testsuites-Management, depending on what is installed.

5.2.8 Keyword: `get_reference_installer_location`

Returns the location of the reference installer. The reference is either the RobotFramework AIO or the Testsuites-Management, depending on what is installed.

5.2.9 Keyword: `get_tsm_version`

Returns the version of the TestsuitesManagement.

5.2.10 Keyword: `get_tsm_version_date`

Returns the version date of the TestsuitesManagement.

5.2.11 Keyword: `get_tsm_app_name`

Returns the application name of the TestsuitesManagement.

5.2.12 Keyword: `get_tsm_installer_location`

Returns the location of the TestsuitesManagement installer.

5.2.13 Keyword: `get_bundle_version`

Returns the version of the entire RobotFramework AIO bundle.

5.2.14 Keyword: `get_bundle_version_date`

Returns the version date of the entire RobotFramework AIO bundle.

5.2.15 Keyword: `get_bundle_name`

Returns the name of the entire RobotFramework AIO bundle.

5.2.16 Keyword: `get_bundle_installer_location`

Returns the location of the installer of the entire RobotFramework AIO bundle.

5.2.17 Keyword: `check_version`

This keyword executes a version check, minversion and maxversion are checked against the referenceversion.

5.2.18 Keyword: `get_status_messages`

Returns the StatusMessages object.

Chapter 6

COnFailureHandle.py

6.1 Class: COnFailureHandle

Imported by:

```
from RobotFramework.TestsuitesManagement.Keywords.COnFailureHandle import  
↪ COnFailureHandle
```

6.1.1 Method: is_noney

Chapter 7

CStruct.py

7.1 Class: CStruct

Imported by:

```
from RobotFramework.TestsuitesManagement.Utils.CStruct import CStruct
```

This `CStruct` class creates the given attributes dynamically at runtime.

Chapter 8

CVersion.py

8.1 Class: enVersionCheckResult

Imported by:

```
from RobotFramework.TestsuitesManagement.Uutils.CVersion import enVersionCheckResult
```

Defines different states that identify the result of the version check

8.2 Class: StatusMessages

Imported by:

```
from RobotFramework.TestsuitesManagement.Uutils.CVersion import StatusMessages
```

Dictionary wrapper for status messages of version check. Needs to use the same keys like defined in enVersionCheckResult

8.2.1 Method: get_messages_dict

Returns the entire messages dictionary.

Background: Robot Framework requires a concrete dictionary, and not a class object containing a dictionary

8.2.2 Method: get

Get a status message by key.

8.2.3 Method: set

Set a status message.

8.3 Class: CVersion

Imported by:

```
from RobotFramework.TestsuitesManagement.Uutils.CVersion import CVersion
```

Validate a user-defined version against a reference version.

The reference can be:

- the version of the TestsuitesManagement (in case of a stand-alone installation)
- the version of the RobotFramework AIO (TestsuitesManagement is part of a bundle)

8.3.1 Method: verifyVersion

This method executes a version check at low level (a token string is returned only).

The `min_version` and the `max_version` are checked against the `reference_version`.

If the `reference_version` is `None` (= not defined by user), the internally defined `bundle_version` (either RobotFramework AIO or TestsuitesManagement) will be used as reference instead.

Arguments:

- `min_version`
/ *Condition*: optional / *Type*: str /
- `max_version`
/ *Condition*: optional / *Type*: str /
- `reference_version`
/ *Condition*: optional / *Type*: str /

Returns:

- `result`
/ *Type*: str /
A token string indicating the result of the vesion check.

8.3.2 Method: checkVersion

This method executes the version check at high level.

The `min_version` and the `max_version` are checked against the `reference_version`.

If the `reference_version` is `None` (= not defined by user), the internally defined `bundle_version` (either RobotFramework AIO or TestsuitesManagement) will be used as reference instead.

The execution includes error messages and exception handling (in opposite to the low level method `verifyVersion`). Impact is that this method influences the execution of the application that calls it.

Arguments:

- `min_version`
/ *Condition*: optional / *Type*: str /
- `max_version`
/ *Condition*: optional / *Type*: str /
- `reference_version`
/ *Condition*: optional / *Type*: str /
- `ext_logger`
/ *Condition*: optional / *Type*: object /
- `status_messages`
/ *Condition*: optional / *Type*: dict /

Returns:

- `True`
/ *Type*: boolean /
Executed version check passed or check not executed.
- `False`
/ *Type*: boolean /
Executed version check failed.

8.3.3 Method: `get_last_error`

Returns the most recently occurred error (during execution of low level method `verifyVersion`)

8.3.4 Method: `bValidateMinVersion`

Static method to validate the required minimum version against the reference version.

Arguments:

- `tReferenceVersion`
/ *Condition*: required / *Type*: tuple /
The version used as reference
- `tMinVersion`
/ *Condition*: required / *Type*: tuple /
The required minimum version

Returns:

- True or False

8.3.5 Method: `bValidateMaxVersion`

Static method to validate the required maximum version against the reference version.

Arguments:

- `tReferenceVersion`
/ *Condition*: required / *Type*: tuple /
The version used as reference
- `tMaxVersion`
/ *Condition*: required / *Type*: tuple /
The required maximum version

Returns:

- True or False

8.3.6 Method: `bValidateSubVersion`

Static method to validate the format of the provided sub-version and parse it into a sub-tuple for version comparison.

Arguments:

- `sVersion`
/ *Condition*: required / *Type*: str /
A part of the entire version string (either the major version, the minor version or the patch version)

Returns:

- `lSubVersion`
/ *Type*: tuple /

8.3.7 Method: tupleVersion

Static method to convert a version string to a tuple of the format: (major, minor, patch)

In case minor/patch version is missing, it is set to 0. E.g: "1" is transformed to "1.0.0" and "1.1" is transformed to "1.1.0"

This method also supports version strings containing additional tags indicating certain types of versions:

- Alpha version: 'a'
- Beta version: 'b'
- Release candidate: 'rc'

Examples: "1.2rc3", "1.2.1b1"

Arguments:

- sVersion
/ *Condition*: required / *Type*: str /
The version string to be converted

Returns:

- lVersion
/ *Type*: tuple /
A tuple which contains the (major, minor, patch) version.

Chapter 9

Event.py

9.1 Class: Event

Imported by:

```
from RobotFramework.TestsuitesManagement.Utils.Events.Event import Event
```

9.1.1 Method: trigger

Chapter 10

ScopeEvent.py

10.1 Class: ScopeEvent

Imported by:

```
from RobotFramework.TestsuitesManagement.Uutils.Events.ScopeEvent import ScopeEvent
```

10.1.1 Method: trigger

10.2 Class: ScopeStart

Imported by:

```
from RobotFramework.TestsuitesManagement.Uutils.Events.ScopeEvent import ScopeStart
```

10.3 Class: ScopeEnd

Imported by:

```
from RobotFramework.TestsuitesManagement.Uutils.Events.ScopeEvent import ScopeEnd
```

Chapter 11

`__init__.py`

11.1 Function: `on`

11.2 Function: `dispatch`

11.3 Function: `register_event`

Chapter 12

LibListener.py

12.1 Class: LibListener

Imported by:

```
from RobotFramework.TestsuitesManagement.Utils.LibListener import LibListener
```

This `LibListener` class defines the hook methods.

- `_start_suite` hooks to every starting testsuite of robot run.
- `_end_suite` hooks to every ending testsuite of robot run.
- `_start_test` hooks to every starting test case of robot run.
- `_end_test` hooks to every ending test case of robot run.

Chapter 13

app_config.py

13.1 Class: AppConfig

Imported by:

```
from RobotFramework.TestsuitesManagement.Utls.app-config import AppConfig
```

Configuration class that contains all the information about the application.

13.1.1 Method: is_robotframework_aio

Returns * True: RobotFramework AIO is installed * False: RobotFramework AIO is not installed (= standalone installation of TestsuitesManagement)

13.1.2 Method: get_package_context_file

Returns path and name of package context file

13.1.3 Method: get_reference_version

Returns the version number used as reference for version checks. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

13.1.4 Method: get_reference_version_date

Returns the version date belonging to the reference. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

13.1.5 Method: get_reference_app_name

Returns the name of the reference application. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

13.1.6 Method: get_reference_installer_location

Returns the location of the reference installer. The reference is either the RobotFramework AIO or the TestsuitesManagement, depending on what is installed.

13.1.7 Method: get_tsm_version

Returns the version of the TestsuitesManagement.

13.1.8 Method: get_tsm_version_date

Returns the version date of the TestsuitesManagement.

13.1.9 Method: get_tsm_app_name

Returns the application name of the TestsuitesManagement.

13.1.10 Method: get_tsm_installer_location

Returns the location of the TestsuitesManagement installer.

13.1.11 Method: get_bundle_version

Returns the version of the entire RobotFramework AIO bundle.

13.1.12 Method: get_bundle_version_date

Returns the version date of the entire RobotFramework AIO bundle.

13.1.13 Method: get_bundle_name

Returns the name of the entire RobotFramework AIO bundle.

13.1.14 Method: get_bundle_installer_location

Returns the location of the installer of the entire RobotFramework AIO bundle.

Chapter 14

app_logger.py

14.1 Function: setup_app_logger

Creates a completely isolated, application-specific logger.
Returns the created logger

Chapter 15

__init__.py

15.1 Class: RobotFramework_TestsuitesManagement

Imported by:

```
from RobotFramework_TestsuitesManagement.__init__ import  
↳ RobotFramework_TestsuitesManagement
```

15.1.1 Method: run_keyword

15.1.2 Method: get_keyword_tags

15.1.3 Method: get_keyword_documentation

15.1.4 Method: failure_occurred

15.2 Class: CConfigLevel

Imported by:

```
from RobotFramework_TestsuitesManagement.__init__ import CConfigLevel
```

15.3 Class: CTestsuitesCfg

Imported by:

```
from RobotFramework_TestsuitesManagement.__init__ import CTestsuitesCfg
```

Chapter 16

Appendix

About this package:

Table 16.1: Package setup

Setup parameter	Value
Name	RobotFrameworkTestsuitesManagement
Version	0.9.0
Date	14.11.2025
Description	Functionality to manage RobotFramework testsuites
Package URL	robotframework-testsuitesmanagement
Author	Mai Dinh Nam Son
Email	son.maidinhnam@vn.bosch.com
Language	Programming Language :: Python :: 3
License	License :: OSI Approved :: Apache Software License
OS	Operating System :: OS Independent
Python required	>=3.0
Development status	Development Status :: 4 - Beta
Intended audience	Intended Audience :: Developers
Topic	Topic :: Software Development

Chapter 17

History

0.1.0	06/2022
<i>Initial version</i>	
0.2.2	07/2022
<i>Created documentation and updated message logs</i>	
0.3.0	07/2022
<i>Added local configuration feature; documentation rework</i>	
0.4.0	03/2023
<i>Maintenance of log output; maintenance of JSON schema validation of configuration file</i>	
0.5.0	03/2023
<i>Added JSON Dotdict feature</i>	
0.5.1	04/2023
<i>Added JSON self-test component</i>	
0.6.0	04/2023
- Reworked the method for loading configuration level 3 - Improved version check method	
0.7.0	05/2023
- Reworked the method for loading configuration level 4 - Improved error message logs - Added a method to check invalid local configuration file	
0.7.1	05/2023
<i>Introduce package context which allows RobotFramework-TestsuitesManagement work in many contexts:</i> - stand-alone - as part of RobotFramework AIO package	
0.7.3	09/2023
<i>New RobotFramework 6.1 adaptation; Maintenance of log output</i>	
0.7.4	12/2023
- Improve dotdict feature - Update logging - Code maintenance	
0.7.5	03/2024
<i>Simplified local configuration loading</i>	
0.7.6	04/2024
- Added naming convention check for parameter names. - Updated logging message.	

0.7.7	04/2024
<i>Extended debugging support. In case of JSON syntax errors in configuration files, the Robot Framework log files contain an extract of the JSON content nearby the position, where the error occurred.</i>	
0.7.8	05/2024
<i>Improved error messages in case of issues while loading the test configuration.</i>	
0.7.9	06/2024
- Updated JSON schema validation. - Resolved redundant code.	
0.7.10	08/2024
- Changed naming convention check behavior of TestsuitesManagement to align with Robot Framework core. - Code maintenance. - Updated documentation. - Improved error message log.	
0.7.11	10/2024
- Fixed missing metadata information in the log while running configuration level 1. - Code improvement. - Improved Maximum and Minimum version check mechanism. - Updated an information for UNKNOWN state.	
0.7.12	2/2025
<i>Fixed bugs and updated error messages of naming convention check feature.</i>	
0.8.0	3/2025
- Added a naming convention check for Robot Framework variables which are set in JSONP configuration files. - Extended logging of test results. - Replaced <code>re</code> package by <code>regex</code> package.	
0.8.1	5/2025
<i>regex module added to the package requirements.</i>	
0.8.2	9/2025
<i>Added the possibility to trigger a version check of the current package (by method <code>versionCheck</code>).</i>	
0.9.0	11/2025
- Rework of version check mechanism. - Added keywords and methods to access several version numbers separately.	